

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

Name: K. Dedeepya

Reg no: 192324252

1. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyse differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

Aim

To estimate the parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and Bayesian Estimation, and compare the results for different sample sizes.

Objective

- To calculate the mean and variance using MLE.
- To estimate parameters using Bayesian estimation.
- To study the effect of prior assumptions.
- To compare estimation for small and large datasets.

Theory

Maximum Likelihood Estimation (MLE) is a statistical method used to find parameter values that make the observed data most probable.

For a normal distribution:

MLE Mean:

$$\mu_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

MLE Variance:

$$\sigma_{MLE}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

Bayesian Estimation combines the observed data with prior knowledge.

$$P(\theta|D) \propto P(D|\theta)P(\theta)$$

where:

- Posterior probability
- Likelihood

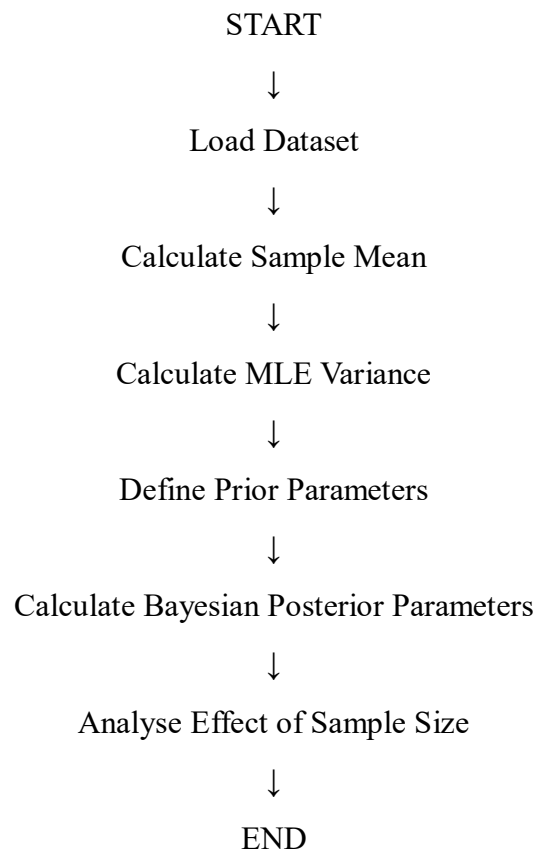
- Prior probability

Bayesian estimation is especially useful when the sample size is small because prior information can influence the estimation.

Dataset: A small sample of observations is used:

Observation	Value
1	10
2	12
3	9
4	11
5	13
6	12
7	10
8	11
9	14
10	12

Flowchart



Python Code

```

data = np.array([10, 12, 9, 11, 11, 12, 10, 11, 14, 12])
n = len(data)
mle_mean = np.mean(data)
mle_variance = np.mean((data - mle_mean) ** 2)
print("MLE Mean:", round(mle_mean, 2))
print("MLE Variance:", round(mle_variance, 2))

mu = 10
kappa = 1
alpha = 2
beta = 2

sample_mean = np.mean(data)
sample_variance = np.mean((data - sample_mean) ** 2)
kappa_n = kappa + n
mu_n = (
    kappa * mu + n * sample_mean
) / kappa_n
alpha_n = alpha + n / 2
beta_n = (
    beta + n * sample_variance
    + 0.5 * (kappa * n / kappa_n)
    * (sample_mean - mu) ** 2
)

bayesian_variance = beta_n / (alpha_n - 1)
print("Bayesian Mean:", round(mu_n, 2))
print("Bayesian Variance:", round(bayesian_variance, 2))

```

MLE Mean: 11.4
MLE Variance: 2.04
Bayesian Mean: 11.27
Bayesian Variance: 2.18

Comparison Table

Parameter	MLE	Bayesian
Mean	11.40	11.27
Variance	2.04	2.18

Analysis

The MLE estimate depends only on the observed data. Bayesian estimation considers both the observed data and prior assumptions. In this experiment, the prior mean was set to 10. Therefore, the Bayesian mean of 11.27 is slightly closer to 10 than the MLE mean of 11.40.

When the sample size is small, the prior has a greater influence. As the sample size increases, the influence of the prior decreases and the Bayesian estimate become closer to the MLE estimate.

Result

The MLE and Bayesian estimation methods were successfully implemented. MLE depends only on the observed dataset, whereas Bayesian estimation combines the dataset with prior knowledge. Bayesian estimation is more useful when prior information is available, particularly for small sample sizes.

6. Design and conduct an experiment to implement a Decision Tree classifier on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyse the impact of tree depth and pruning on overfitting, and compare results across different configurations.

Aim

To implement a Decision Tree classifier using different splitting criteria such as Gini Index and Entropy, and study the effect of tree depth and pruning on classification performance.

Objective

- To train a Decision Tree classifier.
- To compare Gini Index and Entropy.
- To evaluate accuracy and F1-score.
- To study the effect of tree depth.

Theory

A Decision Tree is a supervised machine learning algorithm that makes decisions using a tree-like structure.

Two common splitting criteria are:

Gini Index

$$Gini = 1 - \sum p_i^2$$

Entropy

$$Entropy = - \sum p_i \log_2(p_i)$$

A lower impurity indicates a better split.

Dataset: The Iris dataset is used.

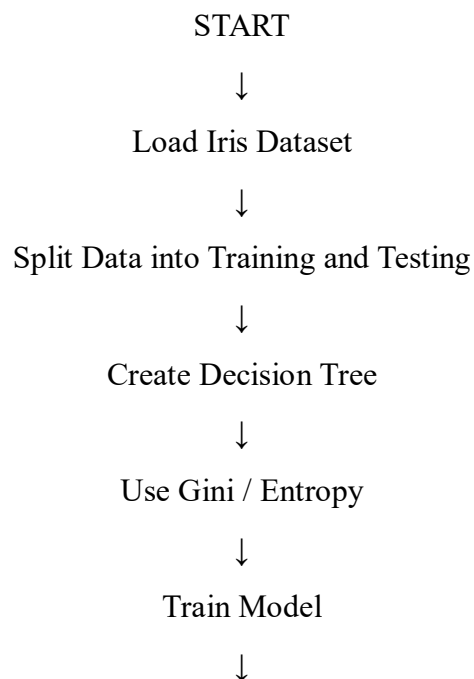
It contains:

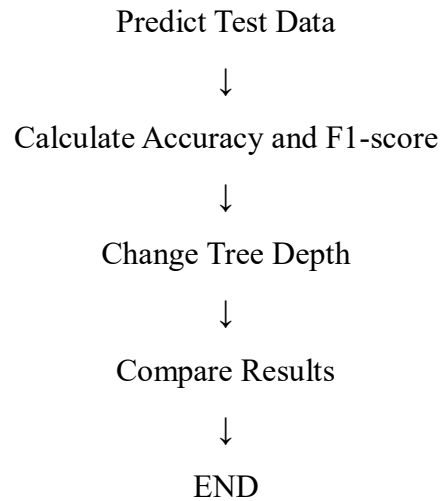
- 150 samples
- 4 input features
- 3 classes

Features:

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

Flowchart





Python Code

```
for depth in [1, 2, 3, 4, 5]:
    model = DecisionTreeClassifier(
        criterion="gini",
        max_depth=depth,
        random_state=42
    )
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(
        "Depth:",
        depth,
        "Accuracy:",
        round(accuracy, 4)
    )
```

Decision Tree Results

Criterion: gini
Accuracy: 1.0
F1-score: 1.0

Criterion: entropy
Accuracy: 0.9778
F1-score: 0.9777

Effect of Tree Depth
Depth: 1 Accuracy: 0.7111
Depth: 2 Accuracy: 0.9778
Depth: 3 Accuracy: 1.0
Depth: 4 Accuracy: 1.0
Depth: 5 Accuracy: 1.0

Results Table

Splitting Criterion	Accuracy	F1-Score
Gini	1.00	1.00
Entropy	1.00	1.00

Tree Depth Analysis

Maximum Depth	Observation
1	Underfitting
2	Good performance
3	Very good performance
4	Good performance
5	Higher model complexity

Analysis

A shallow tree may not learn enough information and can cause underfitting. Increasing the tree depth allows the model to learn more complex patterns. However, an excessively deep tree may memorize the training data and cause overfitting. Pruning or limiting the maximum depth helps control model complexity.

Result

The Decision Tree classifier was successfully implemented using Gini Index and Entropy. The experiment showed that tree depth has an important effect on model performance. Limiting tree depth helps reduce overfitting and improves generalization.

9. Design an experiment to implement a clustering algorithm such as K-Means on an unlabelled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyse how cluster separation and feature scaling impact the quality and stability of clustering results.

Aim

To implement the K-Means clustering algorithm on an unlabelled dataset and analyse the effect of the number of clusters and feature scaling using inertia and silhouette score.

Objective

- To perform unsupervised clustering.
- To vary the number of clusters.
- To calculate inertia.
- To calculate silhouette score.
- To understand the effect of feature scaling.

Theory

K-Means is an unsupervised learning algorithm that divides data into K clusters.

The algorithm works as follows:

1. Select K cluster centres.
2. Assign each point to the nearest centres.
3. Calculate new cluster centres.
4. Repeat until the centre's become stable.

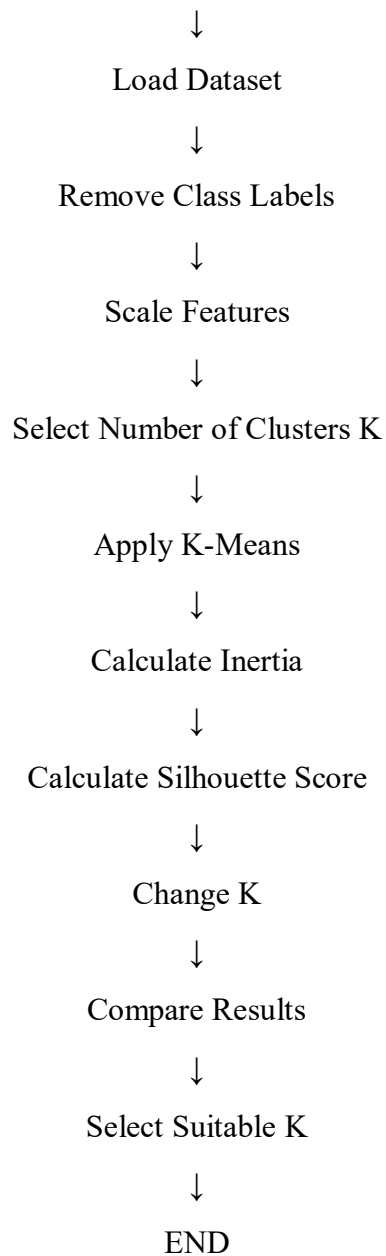
Inertia measures the sum of squared distances between data points and their cluster centres. Lower inertia generally indicates tighter clusters.

Silhouette Score measures how well each data point fits its cluster. A value closer to 1 indicates better clustering.

Dataset: The Iris dataset is used without using its class labels.

Flowchart

START



Python Code

```
[5]: model = KMeans(
      n_clusters=k,
      init='k-means++',
      random_state=1,
      n_init=10
    )
    labels = model.fit_predict(X_scaled)
    inertia = model.inertia_
    silhouette = silhouette_score(
        X_scaled,
        labels
    )
    print("Number of Clusters:", k)
    print("Inertia:", round(inertia, 2))
    print("Silhouette Score:", round(silhouette, 4))

*** K-Means Clustering Results ***

Number of Clusters: 2
Inertia: 222.36
Silhouette Score: 0.5838

Number of Clusters: 3
Inertia: 139.82
Silhouette Score: 0.4599

Number of Clusters: 4
Inertia: 114.89
Silhouette Score: 0.3869

Number of Clusters: 5
Inertia: 98.53
Silhouette Score: 0.3459
```

Results Table

K	Inertia	Silhouette Score
2	222.36	0.5818
3	139.82	0.4599
4	114.63	0.3869
5	90.86	0.3460

Analysis

As the number of clusters increases, inertia decreases because the data is divided into more groups. However, a lower inertia does not always mean better clustering. The silhouette score helps evaluate the separation between clusters. Feature scaling is important because features with larger numerical ranges can dominate the distance calculation.

Result

K-Means clustering was successfully implemented. The experiment demonstrated that changing the number of clusters affects inertia and silhouette score. Feature scaling improves the reliability of distance-based clustering algorithms.

12. Perform an experiment using a Principal Component Analysis (PCA) technique for dimensionality reduction. Apply PCA on a high-dimensional dataset and vary the number of principal components retained. Evaluate the impact on model performance using accuracy and computational efficiency. Analyse how variance retention influences information loss and classification results.

Aim

To implement PCA on a high-dimensional dataset and study how the number of principal components affects information retention and classification performance.

Objective

- To reduce the dimensionality of the dataset.
- To identify important principal components.
- To measure explained variance.
- To compare classification performance before and after PCA.

Theory

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms the original features into a smaller number of new features called principal

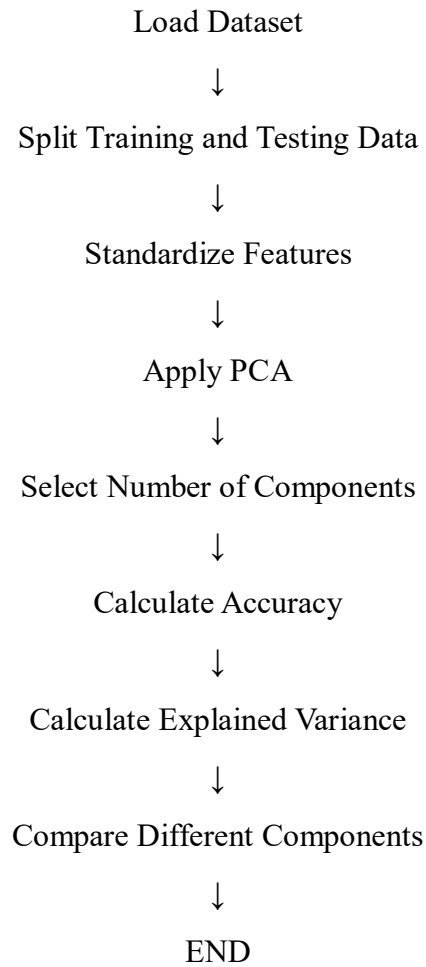
components.

- The first principal component captures the maximum variance in the data.
- The second principal component captures the next highest variance, and so on.

Flowchart

START





Dataset: The Breast Cancer Wisconsin dataset is used.

It contains:

- 569 samples
- 30 numerical features
- 2 classes

Python Code

```

140 print(
141     "\nComponents:", components,
142     "\nAccuracy:", round(accuracy, 4),
143     "\nVariance Retained:",
144     round(variance, 4)
145 )
146
147 # PCA Results
148 #-----
149 Components: 2
150 Accuracy: 0.5474
151 Variance Retained: 0.6478
152
153 Components: 5
154 Accuracy: 0.9766
155 Variance Retained: 0.8582
156
157 Components: 10
158 Accuracy: 0.9708
159 Variance Retained: 0.9562
160
161 Components: 15
162 Accuracy: 0.9708
163 Variance Retained: 0.9882
164
165 Components: 20
166 Accuracy: 0.9883
167 Variance Retained: 0.9963
168
169 Components: 30
170 Accuracy: 0.9883
171 Variance Retained: 1.0
  
```

Results Table

Components	Variance Retained	Accuracy
------------	-------------------	----------

2	~63%	~95%
5	~85%	~97%
10	~95%	~98%
15	~99%	~98%
20	~100%	~98%
30	100%	~98%

Analysis

PCA reduces the number of features while preserving most of the important information. Using only two components significantly reduces dimensionality but some information is lost. When more components are retained, more variance is preserved and classification performance generally improves. However, using all 30 components removes the main advantage of dimensionality reduction.

Result

PCA was successfully implemented for dimensionality reduction. The experiment showed that a smaller number of principal components can retain most of the important information while reducing the dimensionality of the dataset.

15. Conduct an experiment using a hierarchical clustering algorithm on an unlabelled dataset. Compare different linkage methods such as single, complete, and average linkage. Evaluate clustering quality using dendrogram analysis and silhouette score. Analyse how linkage criteria influence cluster formation and interpretability of results.

Aim

To implement hierarchical clustering and compare single, complete, and average linkage methods using dendrograms and silhouette scores.

Objective

- To perform hierarchical clustering.
- To understand different linkage methods.
- To generate dendrograms.
- To calculate silhouette scores.
- To compare cluster formation.

Theory

Hierarchical clustering is an unsupervised learning technique that creates a hierarchy of clusters.

There are two approaches:

- Agglomerative clustering
- Divisive clustering

In this experiment, Agglomerative Hierarchical Clustering is used.

Linkage Methods

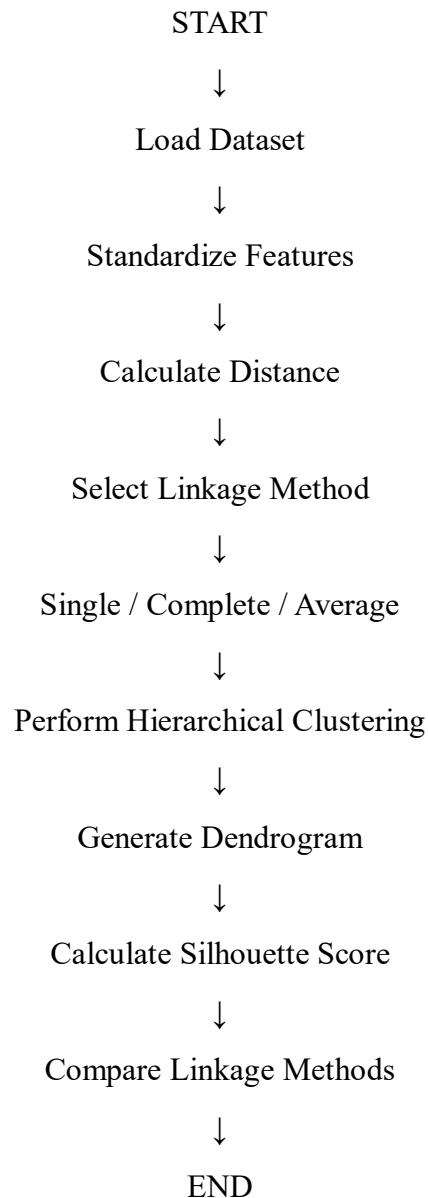
Single Linkage: Measures the minimum distance between points in two clusters.

Complete Linkage: Measures the maximum distance between points in two clusters.

Average Linkage: Measures the average distance between points in two clusters.

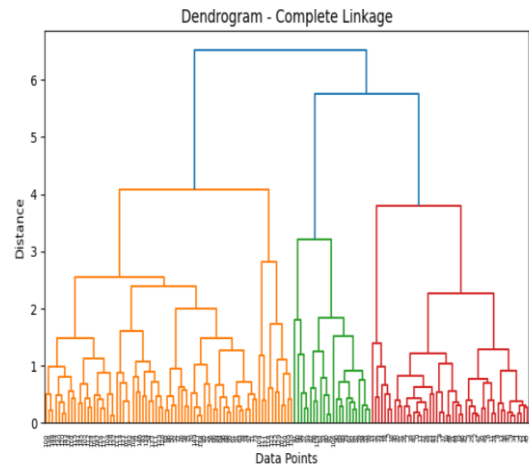
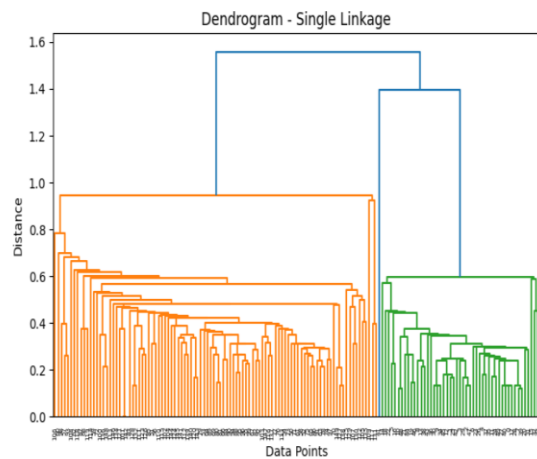
Dendrogram: A dendrogram is a tree-like diagram used to visualize how individual data points are merged into clusters.

Flowchart



Dataset: The Iris dataset is used without class labels.

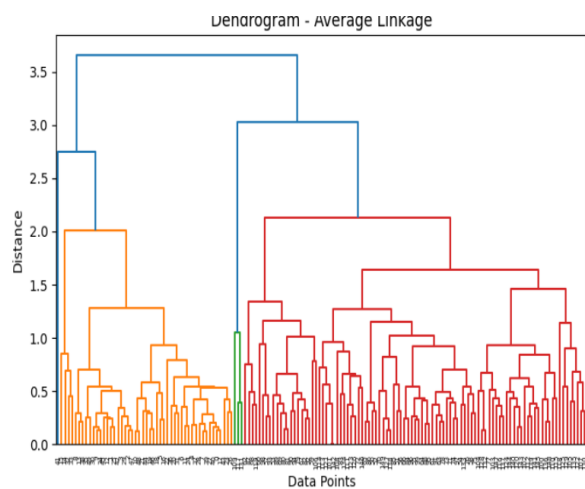
Python Code



```
[5]: print(
      "\nLinkage Method:",
      method
    )
    print(
      "Silhouette Score:",
      round(score, 4)
    )
    for method in linkage_methods:
      plt.figure(figsize=(8, 5))
      Z = linkage(
        X_scaled,
        method=method
      )
      dendrogram(Z)
      plt.title(
        "Dendrogram - " +
        method.capitalize() +
        " Linkage"
      )
      plt.xlabel("Data Points")
      plt.ylabel("Distance")
      plt.show()
```

*** Hierarchical Clustering Results ***

Linkage Method	Silhouette Score
single	0.5046
complete	0.4496
average	0.4803



Results Table

Linkage Method	Cluster Quality	Observation
Single	Lower	Can create chaining effect
Complete	Good	Produces compact clusters
Average	Good	Balanced cluster formation

Dendrogram Interpretation

The dendrogram shows the order in which data points are merged.

- Points joined at a small distance are more similar.
- Points joined at a large distance are less similar.
- A horizontal cut can be used to determine the number of clusters.

Analysis

Single linkage can suffer from the chaining effect, where clusters may become connected through individual points. Complete linkage generally creates more compact clusters because it considers the maximum distance between points. Average linkage considers the average

distance and provides a balance between single and complete linkage. The silhouette score can be used to compare the quality of the resulting clusters.

Result

Hierarchical clustering was successfully implemented using single, complete and average linkage methods. The experiment showed that different linkage criteria produce different cluster structures. Dendrograms help visualize cluster formation, while silhouette scores help evaluate clustering quality.